

Міністерство освіти і науки України
Чернівецький національний університет імені Юрія Федьковича
Навчально-науковий інститут фізико-технічних та комп'ютерних наук
Кафедра комп'ютерних наук

ЗВІТ

до лабораторної роботи № 1

«Постановка задачі та опис предметної області»

з дисципліни «Проектування інтелектуальних інформаційних систем»

Тема: «Self-Healing Pipeline - самовідновлювальний конвеєр
аналізу тональності текстових відгуків»

Виконав: студент 4-го курсу, групи 444а
спеціальності 122 Комп'ютерні науки

Мулик А.В.

Перевірив:

к.ф.-м.н., доцент, Ковальчук М.Л.

Чернівці – 2026

1. ТЕМА ТА НАЗВА СИСТЕМИ

Робоча назва системи: «Self-Healing Pipeline - самовідновлювальний конвеєр аналізу тональності текстових відгуків». Система реалізує повний цикл обробки великих обсягів краудсорсингових текстових даних (на прикладі Yelp Academic Dataset): автоматичне виявлення й виправлення дефектів вхідних записів, класифікацію емоційної тональності відгуків за допомогою локальної великої мовної моделі та надання результатів через вебдашборд моніторингу.

Тема обрана на основі теми кваліфікаційної роботи бакалавра «Розробка самовідновлювального конвеєру ETL та аналітичного сховища для аналізу текстових відгуків з використанням Apache Airflow і локальних великих мовних моделей» і повністю відповідає вимогам лабораторної роботи: система містить власний AI/ML-компонент (NLP-класифікація тональності через LLM), складається з кількох логічних компонентів (конвеєр Apache Airflow, модуль самовідновлення, аналітичне сховище DuckDB, вебдашборд Blazor) та вирішує практично значущу задачу обробки реальних даних.

2. ОПИС ПРЕДМЕТНОЇ ОБЛАСТІ

2.1 Загальна характеристика галузі

Система функціонує на перетині двох суміжних галузей: інженерії даних (Data Engineering / DataOps) - у частині побудови відмовостійких ETL-конвеєрів обробки великих обсягів текстових даних, та обробки природної мови (Natural Language Processing) - у частині автоматичного визначення емоційної тональності текстових відгуків. Система автоматизує повний цикл: імпорт сирих даних у аналітичне сховище, виявлення та виправлення дефектів якості, класифікацію тональності за допомогою мовної моделі, агрегацію статистики та надання результатів через вебінтерфейс моніторингу.

2.2 Актуальність задачі

За оцінками DataOps Impact Assessment 2023, підприємства обробляють у середньому 2,7 петабайти даних на місяць, витрачаючи близько 41% робочого часу інженерів даних на усунення проблем якості - традиційні конвеєри відмовляють «у тиші» без можливості автоматичного відновлення. Водночас реальні краудсорсингові датасети (на прикладі Yelp Academic Dataset) систематично містять дефекти - відсутні значення, хибні типи, порожні чи надмірно довгі тексти, - що унеможлиблює пряму класифікацію без попередньої обробки.

Задача потребує AI, оскільки лексичні словникові методи аналізу тональності (VADER, SentiWordNet) погано справляються з саркастичними та контекстно-залежними текстами (F1 0,55–0,70), а навчання спеціалізованої трансформерної моделі (BERT/RoBERTa) вимагає розміченого доменного датасету, якого немає. Локальна LLM у режимі zero-shot демонструє конкурентний рівень якості (F1 0,75–0,88) та міжочінювацьку надійність, порівнянну з людськими анотаторами, без потреби фінетюнінгу - це й обґрунтовує застосування LLM-підходу замість класичних методів.

2.3 Основні учасники (стейкхолдери)

- Data/Business Analyst - обирає конкретний бізнес із датасету, запускає аналіз його відгуків та переглядає аналітичні звіти й візуалізації.
- Адміністратор / DevOps-інженер - відповідає за розгортання Apache Airflow, конфігурацію DAG-ів та обслуговування інфраструктури конвеєра.
- Ollama (локальний LLM-сервіс) - зовнішній відносно бізнес-логіки сервіс, що надає REST API для класифікації тональності тексту моделлю llama3.2.

- Yelp Academic Dataset - зовнішнє джерело сирих текстових відгуків та метаданих бізнесів для імпорту в систему.

2.4 Вхідні дані системи

- Текстові відгуки користувачів (поле text) разом із рейтингом у зірках (stars) з Yelp Academic Dataset у форматі JSON (обсяг понад 6 ГБ).
- Метадані бізнес-сутностей (назва, адреса, категорія) для пошуку конкретного бізнесу.
- Параметри запуску конвеєра: batch_size, offset, ollama_model.
- Облікові дані користувача для автентифікації (логін/пароль → Beamer-токен).

2.5 Очікуваний результат

Системою на виході формується: (1) класифікована тональність кожного відгуку (POSITIVE/NEGATIVE/NEUTRAL) з оцінкою впевненості (confidence); (2) виправлені (healed) версії дефектних записів із позначкою застосованої стратегії відновлення; (3) структурований health-звіт з інтегральним статусом (HEALTHY/WARNING/DEGRADED/CRITICAL) та детальною статистикою - розподіл тональностей, кореляція зірок і sentiment, середня впевненість за статусами; (4) інтерактивні візуалізації результатів у вебдашборді Blazor (графіки, таблиці, heatmap активності).

3. ПОСТАНОВКА ЗАДАЧІ

3.1 Мета системи

Метою системи є забезпечення надійної, відмовостійкої обробки великих обсягів текстових відгуків із автоматичним виявленням і виправленням дефектів вхідних даних та точною класифікацією їх емоційної тональності без залучення хмарних провайдерів.

3.2 AI/ML-задача

Обрана AI-задача - класифікація тексту (NLP-задача Sentiment Analysis) з трьома класами (POSITIVE/NEGATIVE/NEUTRAL), яка вирішується через zero-shot інференс локальної великої мовної моделі llama3.2 (через Ollama REST API) без донавчання на доменних даних. Вибір обґрунтований відсутністю розміченого доменного датасету для supervised-навчання, перевагами LLM у задачах класифікації без фінетюнінгу (zero/few-shot), що демонструють конкурентну якість і узгодженість із людськими оцінками, а також вимогою конфіденційності - текст відгуків не повинен передаватися зовнішнім хмарним провайдерам.

3.3 Функціональні вимоги

1. Система повинна автоматично виявляти та виправляти щонайменше п'ять типів дефектів вхідного тексту (відсутній текст, хибний тип даних, порожній текст, текст лише зі спецсимволів, надмірно довгий текст) без зупинки обробки пакету.
2. Система повинна підтримувати пакетну обробку даних із конфігурованими параметрами запуску (розмір пакета, зміщення, модель Ollama) через параметризовані DAG Apache Airflow.
3. Система повинна класифікувати тональність кожного відгуку через локальну LLM, повертаючи структуровану відповідь у форматі JSON (sentiment, confidence).
4. Система повинна застосовувати retry-логіку (до трьох спроб) та резервний (fallback) текстовий парсинг відповіді LLM у разі некоректного формату відповіді.
5. Система повинна переходити у режим деградованої обробки (graceful degradation) при недоступності сервісу Ollama, не припиняючи виконання конвеєра.

6. Система повинна формувати структурований health-звіт із чотирма рівнями статусу на основі часток деградованих і відновлених записів.

7. Система повинна надавати вебінтерфейс для перегляду списку запусків, детальної аналітики одного запуску та аудиту дефектних/відновлених записів.

8. Система повинна забезпечувати автентифікацію користувачів на основі Bearer-токенів для доступу до захищених API-ендпоінтів.

3.4 Нефункціональні вимоги

1. Продуктивність: пошукові запити до аналітичного сховища DuckDB по датасету обсягом понад 6 ГБ повинні виконуватися за мілісекунди (порівняно з хвилинами при прямому читанні JSON-файлів).
2. Надійність: відмова зовнішнього сервісу Ollama не повинна призводити до падіння конвеєра - застосовується принцип graceful degradation із безпечними значеннями тональності за замовчуванням.
3. Безпека та конфіденційність: аналіз тональності виконується виключно локальною моделлю без передачі тексту відгуків зовнішнім хмарним провайдерам; доступ до API захищений Bearer-токенами.
4. Масштабованість: обробка датасетів обсягом у мільйони записів повинна підтримуватися через паралельний запуск кількох пакетів (ThreadPoolExecutor) без зміни основної логіки DAG.
5. Супроводжуваність: алгоритм самовідновлення повинен бути детерміністичним і легко розширюваним новими правилами без зміни архітектури конвеєра.

3.5 Очікувані обмеження

- Технічні: локальні моделі малого розміру (1–3 млрд параметрів) демонструють дещо нижчу точність класифікації (F1 0,75–0,88) порівняно з донавченими трансформерними моделями (BERT/RoBERTa, F1 0,87–0,93) -

прийнятний компроміс на користь конфіденційності та відсутності потреби в розміченому датасеті.

- Бізнесові: відсутність доменного розміченого датасету унеможлиблює supervised fine-tuning і пряму кількісну валідацію точності класифікації на достовірних метках (явний ground truth у Yelp-датасеті відсутній, окрім непрямої кореляції з рейтингом у зірках).

- Часові: реалізація обмежена термінами бакалаврської кваліфікаційної роботи, що не дозволило провести повноцінне навантажувальне тестування на мільйонах записів у виробничому середовищі.

4. АНАЛІЗ ІСНУЮЧИХ РІШЕНЬ

4.1 Amazon Comprehend

Хмарний сервіс NLP від AWS із вбудованою функцією аналізу тональності тексту. AI-компонент: попередньо навчені моделі класифікації тональності та виявлення сутностей. Відмінність: Comprehend - платний хмарний сервіс, що вимагає передачі тексту відгуків на сервери AWS (порушення вимоги конфіденційності), і не має вбудованої логіки автоматичного виявлення й виправлення дефектів вхідних даних - передбачається, що дані вже чисті на момент аналізу.

4.2 Great Expectations

Відкритий фреймворк декларативної перевірки якості даних. AI-компонент: статистичне профілювання даних для автоматичної генерації правил-очікувань (expectations). Відмінність: Great Expectations лише фіксує (валідує) порушення якості даних, не виконуючи їх автоматичного виправлення, і не містить жодного компонента аналізу тональності тексту - функціонал обмежений виключно контролем якості без self-healing та без NLP-складової.

4.3 MonkeyLearn / Lexalytics

Комерційні платформи текстової аналітики, що надають API аналізу тональності на основі власних закритих моделей. AI-компонент: класифікатори тональності, навчені на пропрієтарних датасетах постачальника. Відмінність: рішення платні та закриті (vendor lock-in), не підтримують локальний інференс і не інтегрують механізм самовідновлення вхідних даних у єдиний оркестрований конвеєр обробки.

4.4 Узагальнення

Жодний із розглянутих аналогів не поєднує в одній системі автоматичне виправлення дефектів вхідних даних і конфіденційний локальний LLM-аналіз тональності в межах єдиного оркестрованого, відкритого та self-hosted конвеєра. Розроблювана система вирізняється саме цим поєднанням self-healing-логіки на базі Apache Airflow з локальним LLM-інференсом через Ollama, що усуває залежність від хмарних провайдерів і забезпечує повний контроль над конфіденційністю даних.

5. ПОПЕРЕДНЯ СТРУКТУРА СИСТЕМИ

Перелік основних компонентів (попередній варіант, що відповідає реальній архітектурі реалізованого програмного комплексу):

1. Yelp Import Service - імпорт сирих JSON-файлів датасету у аналітичне сховище DuckDB.
2. DuckDB (аналітичне сховище) - колонкове зберігання даних для швидких аналітичних запитів по багатогігабайтному датасету.
3. Apache Airflow DAG-и (self_healing_pipeline, business_analysis_pipeline) - оркестрація шестикрокової послідовності задач обробки (TaskFlow API).
4. Модуль самовідновлення (Self-Healing Module) - діагностика та виправлення п'яти класів дефектів вхідних записів.

5. Модуль аналізу тональності - інтеграція з Ollama REST API (модель llama3.2), retry-логіка та fallback-парсинг.
6. Health-звіт (JSON) - агрегована статистика виконання та інтегральний статус конвеєра.
7. ASP.NET Core Backend (Dashboard) - REST API, сервіс автентифікації, проксі до Airflow REST API.
8. Blazor WebAssembly Client (Dashboard.Client) - вебінтерфейс моніторингу, аналітики та аудиту запусків.
9. SQLite (auth.db) - зберігання облікових записів користувачів для механізму автентифікації.

Орієнтовна схема взаємодії компонентів (текстове подання)

Yelp Academic Dataset (JSON) → Yelp Import Service → DuckDB
DuckDB → Business Service → NDJSON-файл відгуків бізнесу → Airflow DAG
Airflow DAG: diagnose_and_heal_batch (Self-Healing Module) → batch_analyze_sentiment → Ollama REST API (llama3.2) → health_report → JSON-результат
ASP.NET Core Backend ← (читає) JSON-результат, DuckDB → Blazor WASM Client (дашборд)
Користувач → Login → AuthController (Bearer-токен, SQLite) → захищені API-ендпоінти

Наведена структура є попередньою і буде деталізована та уточнена у вигляді формальних Context Diagram (Лабораторна робота № 2) та Component Diagram (Лабораторна робота № 3).

6. ВИСНОВКИ

У межах лабораторної роботи визначено тему та робочу назву інтелектуальної інформаційної системи - Self-Healing Pipeline, що реалізує самовідновлювальний конвеєр обробки текстових відгуків із вбудованим аналізом тональності на основі локальної великої мовної моделі. Описано предметну область на перетині DataOps та NLP, стейкхолдерів, вхідні дані та очікувані результати системи, а також сформульовано мету, тип AI-задачі (zero-shot класифікація тональності тексту через LLM), функціональні й нефункціональні вимоги та обмеження проєкту, зумовлені специфікою локального інференсу й відсутністю доменного розміченого датасету.

Проведено аналіз трьох існуючих аналогів (Amazon Comprehend, Great Expectations, MonkeyLearn/Lexalytics), на основі якого обґрунтовано унікальність поєднання self-healing-логіки та локального конфіденційного LLM-аналізу в межах єдиного відкритого конвеєра. Запропоновано попередню структуру системи з дев'ятьма компонентами, що безпосередньо відповідає реальній архітектурі реалізованого програмного комплексу (Apache Airflow + DuckDB + Ollama/llama3.2 + Blazor) і буде деталізована на наступних етапах курсового проєкту у вигляді Context Diagram, Component Diagram та проєктування ML-підсистеми.